Anexos de la segunda parte

Transcripciones y fragmentos relevantes del experimento

Anexo A Instrucción operacional usada para generar el MVP

```
Implementa el MVP completo siguiendo la propuesta del agente arquitecto. Crea un proyecto ejecutable. Alcance: React H
```

Anexo B Evidencia del código original

```
client/src/App.jsx          SPA, autenticación y panel
client/src/api.js           cliente HTTP
server/src/app.js           rutas, JWT, roles, validación y SQL
server/src/db.js            esquema SQLite y consultas de campañas
server/test/api.test.js     concurrencia, cierre, roles y sesión
```

Restricción de integridad implementada

```
UNIQUE(user_id, campaign_id)
BEGIN IMMEDIATE;
INSERT INTO votes(user_id, campaign_id, burger_id) VALUES (?, ?, ?);
COMMIT;
```

Anexo C Fragmento representativo de la variante A

```
modules/voting/
  domain/vote.js
  application/cast-vote.js
  application/vote-repository.js
  infrastructure/http/vote-router.js
  infrastructure/persistence/pg-vote-repository.js
```

```
class CastVote {
  constructor({ votes, campaigns, ids, clock }) {
    Object.assign(this, { votes, campaigns, ids, clock });
  }
  async execute(input) {
    const campaign = await this.campaigns.findOpenWithOption(input.campaignId, input.optionId);
    if (!campaign) throw new CampaignUnavailableError();
    const created = await this.votes.create({ ...input, id: this.ids.next(), createdAt: this.clock.now() });
    if (!created) throw new DuplicateVoteError();
    return created;
  }
}
```

Anexo D Fragmento representativo de la variante B

```
modules/voting/
  index.js
  http/VoteController.js
  application/VoteService.js
  ports/VoteRepository.js
  infrastructure/PostgresVoteRepository.js
  infrastructure/SqliteVoteRepository.js
  domain/errors.js
```

```
async createUnique(vote) {
  const { rows } = await this.pool.query(
    `INSERT INTO votes (user_id, campaign_id, option_id, created_at)
     VALUES ($1,$2,$3,$4)
     ON CONFLICT (user_id, campaign_id) DO NOTHING
     RETURNING *`,
    [vote.userId, vote.campaignId, vote.optionId, vote.createdAt]
  );
  return rows.length ? { created: true, vote: rows[0] } : { created: false };
}
```

Anexo E Código Mermaid de la arquitectura objetivo

```
flowchart TB
  SPA[React SPA] -->|HTTPS JSON| API[Express API]
  subgraph MONO[Monolito modular]
    AUTH[Usuarios]
    CAT[Catalogo]
    CAMP[Campanas]
    VOTE[Votacion]
    VOTE --> VC[Controller]
    VC --> VS[Service]
    VS --> VR[VoteRepository port]
  end
  API --> AUTH
  API --> CAT
  API --> CAMP
  API --> VOTE
  VR --> PG[(PostgreSQL)]
  VR -. desarrollo .-> SQ[(SQLite)]
```

Anexo F Resultados verificables del MVP

- TC-02: diez solicitudes concurrentes del mismo cliente dejan un único voto.
- TC-01: un voto emitido después del cierre es rechazado.
- TC-03: un usuario CLIENT no puede crear campañas y recibe 403.
- Registro, login y consulta del usuario autenticado funcionan en la prueba de integración.
- La compilación Vite terminó correctamente y el endpoint /api/health respondió ok.